Добавь генерацию API документации в MkDocs

Добавим автоматическую генерацию **API-документации** из docstrings через `mkdocstrings`. Она будет собирать описание функций `rrm.model`, `rrm.scenarios` и `rrm.visualization` прямо из кода.

Обнови mkdocs.yml

Замени содержимое `mkdocs.yml` на:

```
site_name: Responsibility Response Model
site_description: Универсальная модель ответственности, задержки помощи и компенсации
site_author: Your Name
site_url: https://YOUR_USERNAME.github.io/responsibility-response-model/

repo_name: responsibility-response-model
repo_url: https://github.com/YOUR_USERNAME/responsibility-response-model

theme:
  name: material
  language: ru
  features:
    - navigation.tabs
    - navigation.sections
    - navigation.top
    - search.highlight
    - content.code.copy

nav:
  - Главная: index.md
  - Модель: model.md
  - Примеры: examples.md
  - Установка: installation.md
  - API: api.md

plugins:
  - search
  - mkdocstrings:
      default_handler: python
      handlers:
        python:
          options:
            docstring_style: google
            docstring_section_style: table
            show_source: true
            show_root_heading: true
            show_category_heading: true
```

```
            members_order: source
            separate_signature: true
            line_length: 80
            annotations_path: brief

markdown_extensions:
  - admonition
  - pymdownx.highlight
  - pymdownx.superfences
  - pymdownx.arithmatex:
      generic: true
```

`mkdocstrings` поддерживает Google-, NumPy- и Sphinx-стили docstrings; в конфигурации можно задать стиль, формат секций, отображение сигнатур и источников.[1] [2] [3]

Создай docs/api.md

Сохрани как `docs/api.md`:

```
# API Reference

Автоматически сгенерированная документация модуля `rrm`.

## Модель

::: rrm.model
    options:
      members:
        - delay_coefficient
        - support_function
        - model_value
      show_source: true
      show_root_heading: true

## Сценарии

::: rrm.scenarios
    options:
      members:
        - evaluate_scenarios
      show_source: true
      show_root_heading: true

## Визуализация

::: rrm.visualization
    options:
      members:
        - plot_delay_curves
        - plot_scenario_comparison
        - plot_reaction_speed
        - plot_model_principle
        - create_animation
```

```
      show_source: true
      show_root_heading: true
```

Синтаксис `::: rrm.model` указывает `mkdocstrings`, какой Python-объект документировать; опции можно задавать глобально в `mkdocs.yml` или локально для конкретной страницы. [4] [5] [6]

Обнови rrm/model.py

Добавь docstrings в формате Google, чтобы API-документация выглядела аккуратно:

```
def delay_coefficient(t, lam=0.15, critical_time=1.0):
    """Вычисляет коэффициент задержки помощи.

    Args:
        t (float | array-like): Время задержки.
        lam (float): Скорость потери эффективности помощи.
        critical_time (float): Критическое время, до которого помощь считается полной

    Returns:
        float | numpy.ndarray: Коэффициент задержки D(t).

    Examples:
        >>> delay_coefficient(0.5, lam=0.2, critical_time=1.0)
        1.0
    """
    t = np.asarray(t, dtype=float)

    return np.where(
        t <= critical_time,
        1.0,
        np.exp(-lam * (t - critical_time))
    )


def support_function(t, base_support=60.0, decay=0.03):
    """Вычисляет непрерывную поддержку во времени.

    Args:
        t (float | array-like): Время.
        base_support (float): Начальный уровень поддержки.
        decay (float): Скорость уменьшения поддержки.

    Returns:
        float | numpy.ndarray: Уровень поддержки S(t).
    """
    t = np.asarray(t, dtype=float)

    return base_support * np.exp(-decay * t)


def model_value(B, R, S, lam, critical_time=1.0, T=20.0, steps=1000):
    """Вычисляет итоговую ценность решения.
```

```
    Args:
        B (float): Ожидаемая польза.
        R (float): Ожидаемый вред.
        S (float | array-like): Поддержка.
        lam (float): Коэффициент задержки.
        critical_time (float): Критическое время.
        T (float): Горизонт ответственности.
        steps (int): Число шагов интегрирования.

    Returns:
        float: Итоговая ценность V(T).
    """
    t = np.linspace(0, T, steps)
    dt = t[^1] - t[^0]

    D = delay_coefficient(t, lam, critical_time)
    support_integral = np.sum(D * S * dt)

    return B - R + support_integral
```

Обнови rrm/scenarios.py

```
def evaluate_scenarios(T=20.0):
    """Оценивает модель для всех встроенных сценариев.

    Args:
        T (float): Горизонт ответственности.

    Returns:
        dict[str, float]: Словарь с итоговыми значениями V(T).
    """
    results = {}

    for name, params in SCENARIOS.items():
        t = np.linspace(0, T, 1000)

        S = support_function(
            t,
            base_support=params["S_base"],
            decay=params["S_decay"]
        )

        value = model_value(
            B=params["B"],
            R=params["R"],
            S=S,
            lam=params["lambda"],
            critical_time=params["critical_time"],
            T=T
        )

        results[name] = value

    return results
```

Обнови requirements.txt

```
numpy
matplotlib
pillow
pytest
pytest-cov
mkdocs
mkdocs-material
mkdocstrings[python]
```

`mkdocstrings[python]` устанавливает Python-обработчик, который собирает документацию из docstrings.[5] [7]

Обнови GitHub Actions

В `.github/workflows/deploy-docs.yml` уже есть нужная строка:

```
pip install mkdocs mkdocs-material mkdocstrings[python]
```

Оставь её без изменений.

Проверь локально

```
pip install -r requirements.txt
mkdocs serve
```

Открой:

```
http://127.0.0.1:8000/api/
```

Ты увидишь автоматически сгенерированную документацию:

- `delay_coefficient`
- `support_function`
- `model_value`
- `evaluate_scenarios`
- функции визуализации

Проверь сборку

```
mkdocs build --strict
```

Если docstrings или пути указаны неверно, сборка упадёт — это хорошо для CI.

Финальная структура docs

```
docs/
├── index.md
├── model.md
├── examples.md
├── installation.md
└── api.md
```

Финальный принцип

API-документация должна рождаться из кода, а не устаревать рядом с ним.

Теперь при каждом обновлении docstrings в Python-коде MkDocs автоматически перестроит API-справку и опубликует её на GitHub Pages.[8] [9] [4]

⁂

1. https://mkdocstrings.github.io/python/usage/configuration/docstrings/
2. https://mkdocstrings.github.io/usage/handlers/
3. https://mkdocstrings.github.io/python/usage/configuration/signatures/
4. https://mkdocstrings.github.io/mkdocstrings/usage/
5. https://mkdocstrings.github.io/mkdocstrings/
6. https://mkdocstrings.github.io/mkdocstrings/reference/api/
7. https://mkdocstrings.github.io/python/
8. https://mkdocstrings.github.io/python/usage/configuration/general/
9. https://mkdocstrings.github.io/python/usage/
10. https://www.resus.org.uk/professional-library/2025-resuscitation-guidelines/ethics-guidelines
11. https://mkdocstrings.github.io/python/usage/configuration/members/
12. https://mkdocstrings.github.io/python/usage/customization/
13. https://mkdocstrings.github.io/python/usage/configuration/headings/
14. https://mkdocstrings.github.io/mkdocstrings/usage/handlers/
15. https://www.mkdocs.org/user-guide/configuration/
16. https://github.com/mkdocstrings/mkdocstrings/blob/bd8c2bc1918f237a2ae2d00d27ba074071c7e594/docs/handlers/python.md